

ISE 331: Fundamentals of Computer Security

Practice Problems for Lectures 18 to 20 (Software Security)

Directions: Below is a set of exercises and problems you should do for practice. Answering this set of questions should be just one part of your preparation for quizzes and exams. In addition to solving these problems you should:

- Review the lecture notes.
- Review your homework assignments.
- Form a study group and test each other on the material.

Note: The solution is provided on the last page. It is strongly recommended that you work on the questions first before going to the solution.

Part I. Multiple Choice

1. Which is correct regarding Saved UID?
 - A. It is used for all access checks in the operating system.
 - B. It is used to provide root privilege for the current user.
 - C. It is used to separate privileges of the root user.
 - D. It is used to temporarily drop user privileges.
2. Which is NOT a reason that causes privilege escalation?
 - A. Executing as root
 - B. Unnecessary sudoing
 - C. Vulnerabilities in SUID
 - D. Operating System (OS) level vulnerabilities
3. Why is the fundamental reason that Von Neumann architecture is vulnerable to shellcode injection?
 - A. It separates data storage and data execution.
 - B. It shares registers between caller and callee functions.
 - C. It cannot distinguish code and data in memory.
 - D. It can be connected to Input/Output devices.
4. When a function executes the ret (return) instruction, what is the primary action taken by the CPU?
 - A. It subtracts 8 from the stack pointer (rsp).
 - B. It clears all values currently stored in the general-purpose registers.
 - C. It pops the top value of the stack into the instruction pointer (rip).
 - D. It moves the contents of the rbp register into the rsp register.
5. Assume that all the processes have the same root privilege, and they can access all of memory and file systems. Which is incorrect?
 - A. Different processes can clobber each other and mess up each other's data.
 - B. Any process will be able to modify a file or a folder that it does not own.
 - C. Once a malicious program is executed on the machine, the attacker can gain root privilege.
 - D. If only one process is allowed to run at a time, there won't be any harm.

6. Which description best fits sandboxing in the context of software security?
 - A. Untrusted processes execute with almost zero permissions.
 - B. Untrusted processes are blocked from executing unless explicitly permitted.
 - C. Trusted processes are executed with root privilege only.
 - D. Trusted processes are allowed to be executed by default.
7. Which of the following best describes the fundamental difference between Linux Namespaces and Control Groups (cgroups)?
 - A. Namespaces limit how much of a resource a process can use, while cgroups limit what the process can see.
 - B. Namespaces isolate the process's view of resources, while cgroups limit and account for the usage of resources.
 - C. Namespaces are used for filesystem isolation only, while cgroups are used for syscall filtering.
 - D. Namespaces are only available to the root user, while cgroups can be managed by any user.
8. What is the primary purpose of a "Stack Canary" in a compiled program?
 - A. To make the program run more secure by turning on direct memory access.
 - B. To detect if a buffer overflow has overwritten the return address before the function returns.
 - C. To encrypt the local variable data stored in the stack.
 - D. To prevent the user from entering more data than the buffer can hold.
9. One primary security issue with the stack architecture in C is that "control data" and "non-control data" are stored together in the same memory region. Which of the following best illustrates a "control flow hijack" resulting from this design?
 - A. Overwriting a neighboring local integer variable to change the result of a mathematical calculation.
 - B. Overwriting a saved return address on the stack to redirect the program to execute a malicious function.
 - C. Reading past the end of a buffer to discover the value of a secret stack canary.
 - D. Passing a signed integer to a read() function to trigger a buffer overflow.

Part II. True or False

1. To mitigate the problem of privilege escalation, Access Control List (ACL) is used to break down root privileges.
2. In Linux, a child process automatically inherits the user ID (UID) and group ID (GID) of its parent process.
3. Data Execution Prevention (DEP) means that memory regions marked as writable cannot also be marked as executable.
4. Seccomp rules are only enforced on the specific process that loads them and are not inherited by its children.
5. A "hard limit" in a memory cgroup will cause the system to kill a process if it attempts to exceed its assigned memory quota.
6. When using Address Space Layout Randomization (ASLR), the relative distance (offset) between two functions within one executable remains constant.
7. Common programming languages like C, Python, and Java do not automatically check bounds of an array, resulting in Direct Memory Access (DMA) that can be exploited by attackers.
8. If a stack canary is modified by attackers, it cannot provide protection against memory errors anymore.

Part III. Short Answer

1. Why do we want to separate privileged/unprivileged parts of program? Assume you have a program that is separated to two parts, one does GUI-related complex tasks and is larger than the other, the other needs kernel access and is time sensitive. What should their privileges be? Explain.
2. In assembly programming for shellcode, what is the difference between "relative addressing" (e.g., using `[rip+offset]`) and absolute addressing, and why is relative addressing preferred for injected code?
3. Why is it considered a critical security pitfall to call `chroot()` without immediately following it with a `chdir("/")` call?
4. Can an attacker bypass a stack canary by "jumping" over it? If yes, describe a scenario where this might be possible. If no, explain.
5. Why do shellcode developers often want to avoid having null bytes (0x00) in their assembly instructions?

Part I Solution:
1-5 DACCD 6-10 AB BB

Part II Solution:
1-5 FTTFT 6-10 TFF

Part III Solution:

1. Follow the Principle of Least Privilege to minimize the attack surface.
GUI Component should be unprivileged. Because it is large and complex, it is more likely to contain vulnerabilities. Running it without privileges ensures that if the GUI is compromised, the attacker only gains limited access.
Kernel Component should be privileged to perform its specific function. Because it is small, it is easier to make sure that it is secure.
2. Relative addressing uses an offset from the current instruction pointer (rip), while absolute addressing uses a specific memory address.
Relative addressing is preferred because the attacker often cannot predict exactly where in memory the code will be injected. Using relative offsets ensures the code remains functional regardless of its absolute location in the victim process's memory.
3. chroot() only changes the definition of the root directory for the process but does not change the process's current working directory. If the process remains in its original directory outside the "jail," it can still access files in that directory, and the sandboxing becomes useless.
4. Yes. If an attacker can control the index or offset used to write into a buffer, they can skip ("jump over") the memory address containing the canary. For example, an attacker can specify which index of an array to write to.
5. Many common vulnerabilities (like those involving strcpy or gets) use the null byte as a string terminator. If the shellcode contains a 0x00, the vulnerable function will stop copying data at that byte, resulting in an incomplete shellcode.